CMSC216: Bonus Review 2B

Chris Kauffman

Last Updated:
Mon Oct 26 04:56:24 PM EDT 2026

Bonus Review Rules

- 2 Questions will be shown with about 5min per question, 10min total, time limit enforced on Gradescope Quiz
- Each correct answer worth 1 Engagement Point
- Cooperation is allowed and encouraged within your discussion section - *talk to your neighbors, form a class consensus!*
- Questions and Scores posted after all sections have completed bonus review, likely by the following day
- Review is Open Resource just like the exam: https://www.cs.umd.edu/~profk/216/exam-rules.pdf

Staging

- Open up the Gradescope Review Quiz for the day
- Once started, the quiz closes after 10min

Get everybody and their stuff together

Okay...



Question 1

```
1  .text
2  .global minor_test_of_strength
3  minor_test_of_strength:
4    movq $1, %rax
5    movq $1, %rcx
6    testl %eax, %ecx
7    je .RET_1
8    movq $0, %rax
9    ret
10 .RET_1:
11   ret
```

This function will return...

- (A) 1 every time
- (B) 0 every time
- (C) Either 1 or 0 based on its parameter(s) which determine the function behavior
- (D) Either 1 or 0 but it's impossible to say which since the setup and use of the `testl` instruction leads to undefined behavior
- (E) A really large and unpredictable number since moving $1 into `%rax` causes an overflow
- (F) Trick Question: it won't compile as the `testl` instruction uses one register, not two

Question 2

```
==4012== Jump to the invalid address stated on the next line
==4012==    at 0x1320D48: ???
==4012==    by 0x4003F48: CALL_clock_update (test_clock_update_asm.s:227)
==4012==    by 0x4003733: main (test_clock_update.c:668)
==4012==  Address 0x1320d48 is not stack'd, malloc'd or (recently) free'd
==4012==
==4012== Process terminating with default action of signal 11 (SIGSEGV)
```

Which of the following is a likely cause and fix for the above error in Project 3's `clock_update()` function?

- (A) Incorrectly accessing an array location; FIX by adjusting the scaling factor on an instruction of the form
  `... (%reg1, %reg2, scale) ...`
- (B) Incorrect movement size causing data to be read or written that is out of bounds; FIX by adjusting an instruction suffix to select one of `q`/`l`/`w`/`b` that is appropriate to the C data type
- (C) Use of a callee-save register that is not properly restored before returning. FIX by adding a `pushq` instruction at the beginning of the function to save the register and a `popq` instruction to restore it before returning.
- (D) Failure to restore the stack pointer to its original value before returning; FIX by finding the return statement that triggered the problem and using `addq` / `popq` instructions to restore the stack to its original state.
- (E) Improper stack alignment to set up a function call. FIX by adjusting the stack pointer near the beginning of a function with a `subq` instruction to ensure 16-byte alignment accounting for the 8-byte return address on the stack already.
- (F) Incorrect syntax to access a global variable. FIX by adjusting a line like
  `movl myglobal, %eax` TO `movl myglobal(%rip), %eax`
  which uses PC-relative addressing